COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	October 20, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Asajar, Carlo Mari Godino, Jaime Jr.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 ALGORITHM COMPLEXITY

1. PROCEDURE OUTPUT

Tabulated below, are a number of functions against n (from 1 to 10)

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x"

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take 10,000,000,000,000,000,000,000,000,000,000,000 years approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, i.e. 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say that from a computational perspective, the universe is a small thing.

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

Screenshot of Code	**Code: (main_Procedure.cpp)** PROCEDURE > main_Procedure.cpp > experimentalAnalysis() 1 #include <iostream> 2 #include <vector> 3 #include <chrono> 4 #include <iomanip> 5 #include <cstdlib>

```cpp
#include <ctime>
#include <random>
#include <algorithm>
#include "sortingAlgo.h"

// Display all elements in the array
void displayArr(int arr[], int size){

    for(int i = 0; i < size; i++){
        std::cout << arr[i] << " ";

        if((i + 1) % 10 == 0){
            std::cout << std::endl;
        }
    }

    std::cout << std::endl;
}

// Measure the execution time of a sorting algorithm
template <typename SortFunction>
double measureSort(const std::vector<int>& data,
                   SortFunction sortFunction){

    int size = static_cast<int>(data.size());
    std::vector<int> arr = data;

    // Start the timer
    auto begin = std::chrono::steady_clock::now();

    sortFunction(arr.data(), size);

    // Stop the timer
    auto end = std::chrono::steady_clock::now();

    // Verify that the array is sorted
    bool correct = std::is_sorted(arr.begin(), arr.end());

    if(!correct){
        std::cout << "Error: Sorting failed!" << std::endl;
    }

```

```cpp
    // Return the execution time in microseconds
    return std::chrono::duration<double, std::micro>(
        end - begin
    ).count();
}

// Compare Merge Sort and Quick Sort using the same input
void compareAlgorithms(const std::vector<int>& data,
                       const std::string& testName){
```

```
57
58      // Measure Merge Sort execution time
59      double mergeTime = measureSort(
60          data,
61          [](int arr[], int size){
62              mergeSort(arr, size);
63          }
64      );
65
66      // Measure Quick Sort execution time
67      double quickTime = measureSort(
68          data,
69          [](int arr[], int size){
70              quickSort(arr, size);
71          }
72      );
73
74      // Display the experimental results
75      std::cout << std::left << std::setw(20) << testName
76                << std::right << std::setw(18) << mergeTime
77                << std::setw(18) << quickTime
78                << std::endl;
79  }
80
81  // Demonstrate Merge Sort and Quick Sort using 100 elements
82  void sortingDemonstration(){
83
84      const int maxSize = 100;
85
86      int unsortedArr[maxSize];
87      int mergeArr[maxSize];
88      int quickArr[maxSize];
89
90      // Generate random values
91      for(int i = 0; i < maxSize; i++){
92          unsortedArr[i] = rand() % 100;
93      }
94
95      // Copy the original array for each algorithm
96      for(int i = 0; i < maxSize; i++){
97          mergeArr[i] = unsortedArr[i];
98          quickArr[i] = unsortedArr[i];
99      }
```

```
100
101     // Display the original array
102     std::cout << "\nArray:" << std::endl;
103     std::cout << "-------------------------------------------------------"
104               << std::endl;
105
106     displayArr(unsortedArr, maxSize);
107
108     // Sort the array using Merge Sort
```

```
    std::cout << "Merge Sort:" << std::endl;
    std::cout << "-------------------------------------------------------"
              << std::endl;

    mergeSort(mergeArr, maxSize);
    displayArr(mergeArr, maxSize);

    // Sort the array using Quick Sort
    std::cout << "Quick Sort:" << std::endl;
    std::cout << "-------------------------------------------------------"
              << std::endl;

    quickSort(quickArr, maxSize);
    displayArr(quickArr, maxSize);
}

// Perform experimental analysis using different input sizes
void experimentalAnalysis(){

    int sizes[] = {100, 1000, 10000};

    // Initialize the random number generator
    std::mt19937 generator(
        static_cast<unsigned int>(std::time(nullptr))
    );

    std::cout << "\nEXPERIMENTAL ANALYSIS:"
              << std::endl;

    std::cout << "-------------------------------------------------------"
              << std::endl;

    std::cout << std::fixed << std::setprecision(3);

    // Test the algorithms using different input sizes
    for(int size : sizes){

        // Generate randomized input
        std::vector<int> randomData(size);

        for(int i = 0; i < size; i++){
            randomData[i] = i + 1;
        }

        std::shuffle(
            randomData.begin(),
            randomData.end(),
            generator
        );

        // Generate ascending input
        std::vector<int> ascendingData(size);
```

```
161
162         for(int i = 0; i < size; i++){
163             ascendingData[i] = i + 1;
164         }
165
166         // Generate descending input
167         std::vector<int> descendingData = ascendingData;
168
169         std::reverse(
170             descendingData.begin(),
171             descendingData.end()
172         );
173
174         // Display the input size
175         std::cout << "\nInput Size: " << size << std::endl;
176
177         std::cout << "-------------------------------------------------------"
178                   << std::endl;
179
180         // Display the table headings
181         std::cout << std::left << std::setw(20) << "Test Case"
182                   << std::right << std::setw(18)
183                   << "Merge Sort (us)"
184                   << std::setw(18) << "Quick Sort (us)"
185                   << std::endl;
186
187         std::cout << "-------------------------------------------------------"
188                   << std::endl;
189
190         // Test randomized input
191         compareAlgorithms(randomData, "Randomized");
192
193         // Test ascending input
194         compareAlgorithms(ascendingData, "Ascending");
195
196         // Test descending input
197         compareAlgorithms(descendingData, "Descending");
198     }
199 }
```

```
200
201 int main(){
202
203     // Initialize the random seed
204     srand(static_cast<unsigned int>(time(nullptr)));
205
206     // Perform the sorting demonstration
207     std::cout << "\nSORTING ALGORITHM:" << std::endl;
208     std::cout << "-------------------------------------------------------"
209               << std::endl;
210     sortingDemonstration();
211
```

```
    // Perform experimental analysis
    experimentalAnalysis();
    std::cout << std::endl;

    return 0;
}
```

Output

```
SORTING ALGORITHM:
------------------------------------------------------------

Array:
------------------------------------------------------------
90 55 59 20 48 24 92 12 86 44
32 46 41 5 36 25 99 71 89 78
92 81 84 33 58 56 17 30 24 30
65 5 20 93 33 82 7 78 42 83
2 33 18 99 82 78 80 12 65 50
10 6 3 34 29 70 6 39 11 74
46 79 45 58 40 51 24 24 85 85
93 95 5 76 93 20 20 37 34 25
45 0 31 23 14 32 20 7 71 59
14 23 46 9 64 45 97 43 2 88

Merge Sort:
------------------------------------------------------------
0 2 2 3 5 5 5 6 6 7
7 9 10 11 12 12 14 14 17 18
20 20 20 20 20 23 23 24 24 24
24 25 25 29 30 30 31 32 32 33
33 33 34 34 36 37 39 40 41 42
43 44 45 45 45 46 46 46 48 50
51 55 56 58 58 59 59 64 65 65
70 71 71 74 76 78 78 78 79 80
81 82 82 83 84 85 85 86 88 89
90 92 92 93 93 93 95 97 99 99
```

```
Quick Sort:
------------------------------------------------------------
0 2 2 3 5 5 5 6 6 7
7 9 10 11 12 12 14 14 17 18
20 20 20 20 20 23 23 24 24 24
24 25 25 29 30 30 31 32 32 33
33 33 34 34 36 37 39 40 41 42
43 44 45 45 45 46 46 46 48 50
51 55 56 58 58 59 59 64 65 65
70 71 71 74 76 78 78 78 79 80
81 82 82 83 84 85 85 86 88 89
90 92 92 93 93 93 95 97 99 99


EXPERIMENTAL ANALYSIS:
------------------------------------------------------------

Input Size: 100
------------------------------------------------------------
Test Case              Merge Sort (us)   Quick Sort (us)
------------------------------------------------------------
Randomized                      49.500            4.600
Ascending                       45.100           16.600
Descending                      43.500           12.400

Input Size: 1000
------------------------------------------------------------
Test Case              Merge Sort (us)   Quick Sort (us)
------------------------------------------------------------
Randomized                     483.300           62.300
Ascending                      439.800         1483.400
Descending                     447.900          897.300

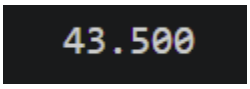
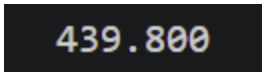
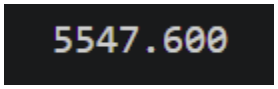
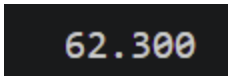
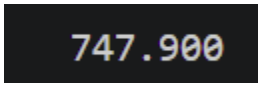
Input Size: 10000
------------------------------------------------------------
Test Case              Merge Sort (us)   Quick Sort (us)
------------------------------------------------------------
Randomized                    5883.400          747.900
Ascending                     6314.600       146638.000
Descending                    5547.600        84110.300
```

Show your answer and analysis in the table.
(Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools)

	Best Case	Worst Case	Analysis
Algorithm 1: Merge Sort	O(n log n)	O(n log n)	It divides the values into smaller parts and then combines them in sorted order. Its performance stays almost the same even if the original values are already sorted or arranged differently
Algorithm 2: Quick Sort	O(n log n)	$O(n^2)$	$O(n^2)$ It works faster when the pivot divides the values into balanced parts. However, when the parts become too unbalanced, it needs more time to finish sorting, which can make its performance slower.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

Show your answer and analysis in the table.
(Table 14-2. Best- and Worst-Case Analysis using Experimental Tools)

Input Size (N)	Algorithm 1: Merge Sort	Algorithm 2: Quick Sort
Best Case	100  1,000  10,000 	100  1,000  10,000 

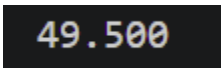
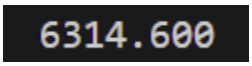
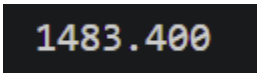
Worst Case	100  1,000  10,000  	100  1,000  10,000 

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Analysis:	
Theoretical Analysis	Merge Sort has O(n log n) time complexity in both the best and worst cases. This means that it can handle larger amounts of data without having a very large increase in execution time. On the other hand, Quick Sort has O(n log n) in the best case and $O(n^2)$ in the worst case. It can become slower when the pivot keeps dividing the values into unbalanced parts.
Experimental Analysis	Based on the results, Quick Sort performed faster than Merge Sort when the values were arranged randomly. However, when the values were already in ascending or descending order, its execution time became much higher, especially with 10,000 elements. Meanwhile, Merge Sort showed more consistent results because its execution time did not change as much when the arrangement was different.
Analysis and Comparison	Both algorithms successfully arranged the values in the correct order, but their performance was different depending on the input. Quick Sort was faster when the values were randomized, while Merge Sort gave more consistent execution times for all tested arrangements. This shows that Quick Sort can be a good choice when its pivot divides the values properly, but Merge Sort is more reliable when a consistent performance is needed.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis
- Time and Space Complexities

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

Pseudocode

Linear Search

```
LINEAR_SEARCH(array, target, n)
   FOR i = 0 TO n - 1
      IF array[i] == target
         RETURN i
   RETURN -1
```

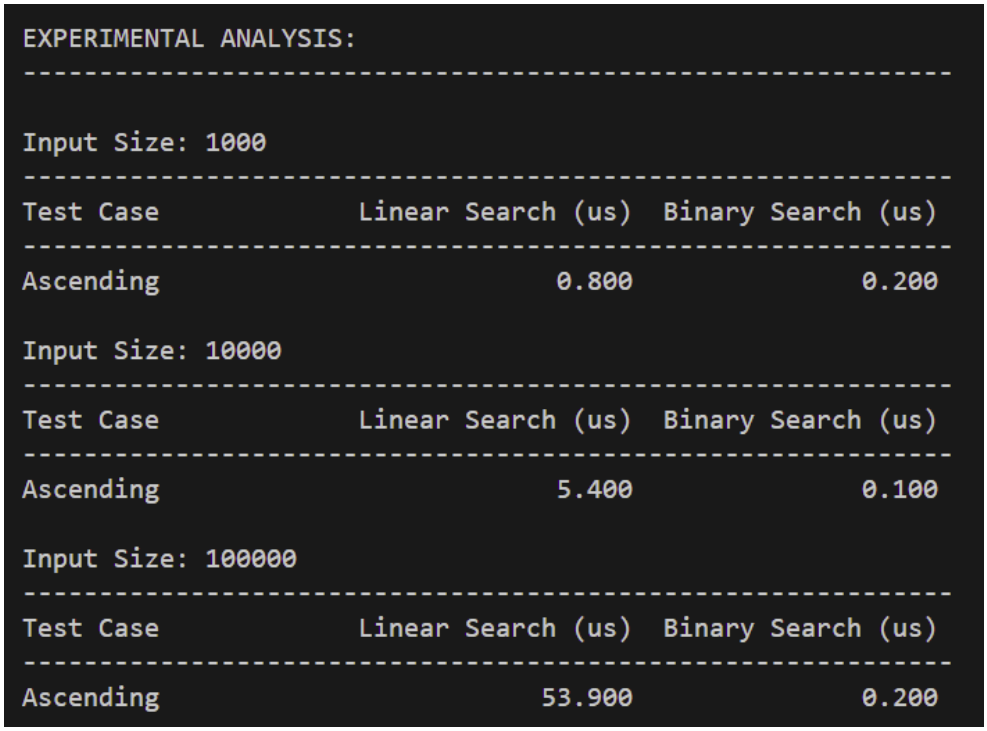
Binary Search:

```
BINARY_SEARCH(array, target, n)
   low = 0
   high = n - 1

   WHILE low <= high
      mid = low + (high - low) / 2

      IF array[mid] == target
         RETURN mid
      ELSE IF array[mid] < target
         low = mid + 1
      ELSE
         high = mid - 1

   RETURN -1
```

Output	
Observation	Linear Search checks each value one by one, so it can take longer when the target is near the end or not present. Binary Search is faster for large datasets because it divides the search range into half in every step. Both algorithms use O(1) extra space in the iterative versions, but Binary Search requires the values to be sorted first.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1 000	0.200	0.800
10,000	0.100	5.400
10,000	0.200	53.900

Provide your analysis.

Analysis:	
Theoretical Analysis	Linear Search checks the values one by one, so its worst-case time complexity is O(n). Binary Search divides the search range into half in every step, giving it a worst-case time complexity of O(log n). Both use O(1) extra space in the iterative array implementation. However, Binary Search needs the values to be sorted before searching.
Experimental Analysis	Based on the results, Binary Search was faster than Linear Search in all three input sizes. When the input increased, Linear Search also took more time because it checked all the values when the target was not found. Meanwhile, Binary Search remained fast because it kept reducing the search range by half. The small changes in its recorded time can happen because the measurements are very short.
Analysis and Comparison	Both algorithms can search for a target, but they work differently. Linear Search is simple and can be used with unsorted values, but it can become slower when the input gets larger. Binary Search is more efficient for large datasets, but the values must be sorted first. Overall, Binary Search is better for large sorted datasets, while Linear Search is still useful for small or unsorted datasets.

Asajar's Conclusion:

The activity gave a better understanding of how the performance of an algorithm can change depending on the input size and how its runtime can be analyzed using theoretical and experimental methods. For the lessons learned, Merge Sort has an O(n log n) runtime in both the best and worst cases, while Quick Sort has O(n log n) in the best case and $O(n^2)$ in the worst case when the pivot divides the values into unbalanced parts. For the supplementary activity, Linear Search checks the values one by one and has an O(n) worst-case runtime, while Binary Search has O(log n) because it divides the search range into half in every step. For the procedure, the algorithms were tested using different input sizes, then the execution times were recorded and compared with their expected runtimes. These results helped show how the input size and data arrangement can affect the performance of each algorithm.

One difficulty faced was understanding how the theoretical complexity connects with the actual execution time, especially when the recorded times were very small and changed between tests. Another difficulty was understanding why Quick Sort can become slower when the pivot does not divide the values properly and why Binary Search needs sorted data before searching. Even with these difficulties, the activity helped explain the differences between the algorithms and their performance. Overall, the activity was completed with a better understanding of algorithm analysis, but there is still room for improvement in interpreting experimental results and understanding the different complexity cases. More practice with coding, testing, and comparing algorithms can help improve the ability to choose the most suitable algorithm for different problems.

Godino's Conclusion:

In this activity, I learned how to analyze the time and space complexity of different algorithms and understand how the input size affects their performance. In the procedure, Merge Sort and Quick Sort were compared to see how they perform with different input sizes and data arrangements. I learned that Merge Sort has a consistent time complexity of O(nlogn), while Quick Sort can reach On^2 when the pivot selection is not good. In the supplementary activity, Binary Search and Linear Search were also compared, and I understood that Binary Search is usually faster for large datasets because it divides the search range into smaller parts, while Linear Search checks the elements one by one. Overall, I was able to understand why choosing an appropriate algorithm is important for making a program more efficient. However, there is still room for improvement in analyzing the results and understanding the different complexity cases. With more practice in coding and testing algorithms, I can improve my skills and become better at choosing the most suitable algorithm for different problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.